

Go for Senior .NET Engineers

Phase 7: Mindset Shifts

6 Lessons: SOLID in Go, No ORM, Error Verbosity, Package Design, Composition, Go Proverbs

The Final Phase — From C# Thinking to Go Thinking

Lesson 60: SOLID Principles — How They Map to Go

SOLID principles still apply in Go, but they manifest very differently than in C#. Go has no classes, no abstract base types, and no explicit interface implementation. Yet idiomatic Go code naturally satisfies SOLID — often better than heavily-patterned C# code.

60.1 — Single Responsibility (SRP)

C# / .NET Thinking

```
// C# – Separate classes
class UserValidator { ... }
class UserRepository { ... }
class UserService {
    UserValidator _validator;
    UserRepository _repo;
    // orchestrates both
}
```

Go Thinking

```
// Go – Same idea, different shape
// Separate packages or files
func ValidateUser(u User) error { ... }

type UserRepo struct { db *sql.DB }
func (r *UserRepo) FindByID(...)

// Service composes them
// No class hierarchy needed
```

MINDSET SHIFT: In Go, SRP is about packages and functions, not classes. A Go package IS the unit of responsibility. Keep packages small and focused.

60.2 — Open/Closed (OCP)

```
// In C#, you extend via inheritance or virtual methods
// In Go, you extend via interfaces and composition

// Define behavior as an interface
type Notifier interface {
    Notify(ctx context.Context, msg string) error
}

// Add new notification types without modifying existing code
type EmailNotifier struct { ... }
func (e *EmailNotifier) Notify(ctx context.Context, msg string) error { ... }

type SlackNotifier struct { ... }
func (s *SlackNotifier) Notify(ctx context.Context, msg string) error { ... }

// New: SMS, Push, Webhook – just implement Notifier
// No base class to modify, no virtual override
```

60.3 — Liskov Substitution (LSP)

In Go, LSP is enforced at compile time through interfaces. If a type implements an interface, it can be substituted anywhere that interface is accepted. There is no fragile base class problem because there are no base classes.

60.4 — Interface Segregation (ISP)

```
// C# temptation: large interfaces
// IRepository<T>: GetAll, GetById, Create, Update, Delete,
//   GetPaged, Search, BulkInsert, SoftDelete, ...

// Go idiom: SMALL interfaces, defined by the consumer
type UserFinder interface {
    FindByID(ctx context.Context, id string) (*User, error)
}

type UserCreator interface {
    Create(ctx context.Context, u *User) error
}

// A handler only depends on what it needs:
func NewGetUserHandler(finder UserFinder) http.Handler { ... }
func NewCreateUserHandler(creator UserCreator) http.Handler { ... }

// The concrete repo implements BOTH, but consumers
// only see the methods they need
```

MINDSET SHIFT: Go proverb: 'The bigger the interface, the weaker the abstraction.' Start with 1-2 method interfaces. `io.Reader` (1 method) is the most powerful interface in Go. Define interfaces where they are CONSUMED, not where they are implemented.

60.5 — Dependency Inversion (DIP)

```
// In C# you register services in DI container:
// services.AddScoped<IUserRepo, PostgresUserRepo>();

// In Go, pass dependencies explicitly (constructor injection):
type OrderService struct {
    repo    OrderRepository // interface
    notify  Notifier          // interface
    logger  *slog.Logger
}

func NewOrderService(r OrderRepository, n Notifier, l *slog.Logger) *OrderService {
    return &OrderService{repo: r, notify: n, logger: l}
}

// In main(), wire everything explicitly:
func main() {
    db := connectDB()
    repo := postgres.NewOrderRepo(db)
    notifier := slack.NewNotifier(webhookURL)
    logger := slog.Default()
```

```
    svc := NewOrderService(repo, notifier, logger)
    // ...
}
```

TIP: Most Go projects use 'manual wiring' in `main()`. No DI container. This is intentional: the dependency graph is visible, debuggable, and doesn't require learning a framework. For very large projects, `google/wire` (compile-time DI) or `uber-go/fx` are options, but they are rarely needed.

Lesson 61: No ORM Love — database/sql, sqlx, sqlc

The Go community is skeptical of ORMs. Unlike .NET where Entity Framework Core dominates, Go developers generally prefer explicit SQL. The motto: 'Write SQL. Know your queries.' This is one of the biggest culture shocks for C# developers.

61.1 — The Go Data Access Spectrum

Tool	Approach
database/sql (stdlib)	Raw SQL, manual Scan() into structs
sqlx	database/sql + struct scanning, named params
sqlc	Write SQL, generate type-safe Go code
Bun / Ent	ORM-like (less popular, used selectively)
GORM	Full ORM (popular but controversial in Go)

61.2 — EF Core vs Go Approaches

C# / .NET Thinking

```
// C# – EF Core
var users = await ctx.Users
    .Where(u => u.Age > 18)
    .OrderBy(u => u.Name)
    .ToListAsync();

// Migration: dotnet ef migrations add
// Schema: generated from C# models
// Queries: translated from LINQ to SQL
```

Go Thinking

```
// Go – sqlc (most idiomatic)
// Write SQL in .sql files:
// -- name: ListAdultUsers :many
// SELECT * FROM users
// WHERE age > $1
// ORDER BY name;

// sqlc generates type-safe Go:
users, err := queries.ListAdultUsers(
    ctx, 18)
```

61.3 — sqlc: Write SQL, Generate Go

```
// 1. Write SQL in query.sql:
-- name: GetUser :one
SELECT id, name, email FROM users WHERE id = $1;

-- name: CreateUser :one
INSERT INTO users (name, email)
VALUES ($1, $2) RETURNING *;

// 2. Run: sqlc generate
// 3. Use generated code:
user, err := queries.GetUser(ctx, userID)
if err != nil {
```

```
    if errors.Is(err, sql.ErrNoRows) {
        return nil, ErrNotFound
    }
    return nil, fmt.Errorf("get user: %w", err)
}

// No reflection, no magic, no N+1 surprises
// The SQL IS the source of truth
```

61.4 — Migrations

```
// Go migration tools (no EF-style auto-migration):
// - golang-migrate/migrate (SQL files, CLI + library)
// - pressly/goose (SQL or Go-based migrations)
// - atlas (modern, declarative schema management)

// Example with goose:
// migrations/001_create_users.sql
-- +goose Up
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    created_at TIMESTAMPTZ DEFAULT now()
);

-- +goose Down
DROP TABLE users;

// Run: goose -dir migrations postgres "connstring" up
```

MINDSET SHIFT: Stop thinking 'which ORM should I use?' and start thinking 'I write SQL.' The Go community believes that SQL is already a great query language — there is no need to invent another abstraction on top of it. sqlc gives you type safety without hiding the SQL.

Lesson 62: Embracing Error Verbosity

The most common complaint from C# developers learning Go: 'if err != nil is everywhere!' This lesson explains why Go chose explicit error handling, how to do it well, and why it actually leads to more reliable software.

62.1 — The C# Exception Model vs Go Error Model

C# / .NET Thinking

```
// C# - Exceptions (invisible flow)
try {
    var user = await GetUser(id);
    var order = await CreateOrder(user);
    await SendConfirmation(order);
} catch (UserNotFoundException) {
    // handle
} catch (OrderException ex) {
    // handle
} catch (Exception ex) {
    // catch-all
}

// Problem: any line can throw
// You cannot tell which calls fail
// by reading the happy path
```

Go Thinking

```
// Go - Errors (explicit flow)
user, err := GetUser(ctx, id)
if err != nil {
    return fmt.Errorf("get user: %w", err)
}

order, err := CreateOrder(ctx, user)
if err != nil {
    return fmt.Errorf("create order: %w",
err)
}

if err := SendConfirmation(ctx, order);
err != nil {
    return fmt.Errorf("send confirm: %w",
err)
}
// Every failure point is visible
```

62.2 — Why Go Chose This

Go's designers observed that most exception-based code in production has poor error handling: catch blocks that log and swallow, blanket catch(Exception), or no catch at all (letting errors propagate silently). By making errors explicit return values, Go forces developers to make a conscious decision at every failure point.

62.3 — Patterns to Reduce Verbosity

```
// Pattern 1: Early return (guard clauses)
func processOrder(ctx context.Context, id string) error {
    order, err := repo.Find(ctx, id)
    if err != nil {
        return fmt.Errorf("find order %s: %w", id, err)
    }
    // happy path continues unindented
    return ship(ctx, order)
}

// Pattern 2: Helper for repetitive I/O
type errWriter struct {
```

```
    w    io.Writer
    err error
}
func (ew *errWriter) write(data []byte) {
    if ew.err != nil { return }
    _, ew.err = ew.w.Write(data)
}
// Use: ew.write(header); ew.write(body); return ew.err

// Pattern 3: errgroup for parallel operations
g, ctx := errgroup.WithContext(ctx)
g.Go(func() error { return fetchUser(ctx) })
g.Go(func() error { return fetchOrders(ctx) })
if err := g.Wait(); err != nil { return err }
```

MINDSET SHIFT: Stop resenting 'if err != nil'. Start seeing it as documentation. Each error check tells you: 'this operation can fail, and here is what we do about it.' After a few months, you will find C# try/catch blocks feel like they are hiding important information.

Lesson 63: Package Design — Flat Structure & Avoiding Over-Engineering

C# developers tend to create deep folder hierarchies: Controllers, Services, Repositories, Models, DTOs, Interfaces, Extensions. Go projects are flatter. The standard layout for a Go service is deliberately simple.

63.1 — .NET Clean Architecture vs Go Layout

C# / .NET Thinking

```
// C# – Clean Architecture
src/
  MyApp.Api/
    Controllers/
    Middleware/
  MyApp.Application/
    Services/
    Interfaces/
    DTOs/
  MyApp.Domain/
    Entities/
    ValueObjects/
    Events/
  MyApp.Infrastructure/
    Persistence/
    Repositories/
    ExternalServices/
```

Go Thinking

```
// Go – Typical service layout
cmd/
  myservice/
    main.go
internal/
  user/
    handler.go
    service.go
    repo.go
    model.go
  order/
    handler.go
    service.go
    repo.go
pkg/
  httputil/
    middleware.go
go.mod
```

63.2 — Key Layout Conventions

Directory	Purpose
cmd/	Entry points (main packages). One per binary.
internal/	Private packages. Cannot be imported by other modules.
pkg/	Public library packages (optional, some teams skip this)
<feature>/	Group by domain feature, not by technical layer
go.mod	Module definition (root of the project)

63.3 — The internal/ Firewall

```
// internal/ is enforced by the Go compiler
// Code in internal/ CANNOT be imported by other modules
```

```
// This is Go's equivalent of C# internal access modifier
// but at the package level, enforced by the toolchain

module github.com/org/myservice

// Other modules CANNOT import:
// github.com/org/myservice/internal/user

// But CAN import:
// github.com/org/myservice/pkg/httputil
```

63.4 — Package Design Rules

```
// 1. Name packages by what they PROVIDE, not what they CONTAIN
//   Good: package user, package order, package httputil
//   Bad:  package models, package utils, package helpers, package common

// 2. Avoid package names that stutter with their contents
//   Good: user.Service, http.Client
//   Bad:  user.UserService, http.HTTPClient

// 3. A package should have ONE clear purpose
//   The user/ package handles users: model, repo, service, handler
//   NOT: user model in one package, user repo in another

// 4. Start flat, split only when pain appears
//   Begin with everything in one package
//   Extract packages when a clear boundary emerges
```

MINDSET SHIFT: In C#, you architect first: layers, abstractions, interfaces, DI registrations. In Go, you start with one package and one file. You split when it hurts, not before. 'A little copying is better than a little dependency' — Go Proverb.

Lesson 64: Composition Over Inheritance — Go's Core Design Philosophy

Go has no classes, no inheritance, no virtual methods, no abstract base types. This is not a limitation — it is a deliberate design choice. Go proves that you can build large, maintainable systems using only composition, interfaces, and embedding.

64.1 — The C# Way vs The Go Way

C# / .NET Thinking

```
// C# – Inheritance hierarchy
abstract class Shape {
    public abstract double Area();
    public virtual string Describe() =>
        $"Shape with area {Area():F2}";
}

class Circle : Shape {
    public double Radius { get; }
    public override double Area() =>
        Math.PI * Radius * Radius;
}

class Square : Shape {
    public double Side { get; }
    public override double Area() =>
        Side * Side;
}
```

Go Thinking

```
// Go – Interface + composition
type Shape interface {
    Area() float64
}

type Circle struct {
    Radius float64
}

func (c Circle) Area() float64 {
    return math.Pi * c.Radius * c.Radius
}

type Square struct {
    Side float64
}

func (s Square) Area() float64 {
    return s.Side * s.Side
}

// No base class, no override keyword
```

64.2 — Shared Behavior Without Inheritance

```
// C# approach: base class with shared logic
// Go approach: embed a helper struct

// Shared logging behavior
type withLogger struct {
    logger *slog.Logger
}

func (l *withLogger) logInfo(msg string, args ...any) {
    l.logger.Info(msg, args...)
}

// Embed into services that need logging
type UserService struct {
    withLogger
    repo UserRepository
}

type OrderService struct {
    withLogger
```

```
    repo OrderRepository
}

// Both can call: svc.logInfo("something happened")
// But there is no fragile base class – withLogger is just data
```

64.3 — Why No Inheritance Works

Inheritance Problem	Go's Solution
Fragile base class	No base classes – embedding is explicit
Diamond problem	No multiple inheritance – compose explicitly
Deep hierarchies	Flat composition, 1-2 levels max
Virtual dispatch overhead	Interface dispatch (single level)
Abstract class lock-in	Small interfaces, defined by consumer
God objects	Many small types composed together

MINDSET SHIFT: Stop looking for where to put your base class. In Go, there is no base class. Ask instead: 'What interface does this consumer need?' and 'What small struct can I embed to share behavior?' These two questions replace the entire OOP inheritance design process.

Lesson 65: Go Proverbs — The Philosophy Behind the Language

Rob Pike's 'Go Proverbs' (go-proverbs.github.io) capture the design philosophy. As a senior .NET engineer, understanding these proverbs will help you write idiomatic Go instead of 'C# in Go syntax.'

65.1 — Key Proverbs, Translated for C# Developers

"Don't communicate by sharing memory, share memory by communicating."

C# instinct: shared state + lock. Go instinct: send data through channels. Instead of a `ConcurrentDictionary` protected by a lock, use a goroutine that owns the data and communicates via channels.

"The bigger the interface, the weaker the abstraction."

C# instinct: `IRepository<T>` with 15 methods. Go instinct: one-method interfaces like `io.Reader`, `fmt.Stringer`, `error`. The standard library's most powerful interfaces have 1-2 methods.

"A little copying is better than a little dependency."

C# instinct: create a `Shared.Utilities` NuGet package. Go instinct: copy 10 lines of code into your package rather than adding a dependency. Dependencies have maintenance costs that often exceed the cost of duplication.

"Make the zero value useful."

C# instinct: require constructor initialization. Go instinct: design structs so they work correctly when all fields are zero. `sync.Mutex` is usable without initialization. `bytes.Buffer` is empty and ready to use. Your types should be too.

"Clear is better than clever."

C# instinct: fluent chains, LINQ one-liners, expression trees. Go instinct: straightforward code that a new team member can read in 30 seconds. Go culture actively discourages 'clever' code.

"Errors are values."

C# instinct: exceptions for control flow. Go instinct: errors are regular values you can store, pass, compare, and compose. They are not exceptional — they are expected.

"Don't just check errors, handle them gracefully."

C# instinct: catch and log. Go instinct: wrap with context (`fmt.Errorf("%w")`), return to caller, let the top-level handler decide what to do. Every layer adds context, none swallow the error.

65.2 — The Simplicity Manifesto

```
// Go's core design principles:
// 1. Simplicity over expressiveness
//   → No operator overloading, no implicit conversions
// 2. Composition over inheritance
//   → No classes, no virtual methods
// 3. Explicit over implicit
//   → No hidden control flow (exceptions)
// 4. One way to do things
//   → gofmt, one loop keyword (for)
// 5. Readability over writability
//   → Code is read 10x more than written
// 6. Standard library over frameworks
//   → net/http, encoding/json, log/slog
// 7. Small interfaces over large abstractions
//   → io.Reader, error, fmt.Stringer
```

65.3 — Your 30-Day Transition Checklist

```
// Week 1: Stop fighting the language
// ☐ Accept gofmt (tabs, brace placement)
// ☐ Accept if err != nil (it's not going away)
// ☐ Accept no generics for everything (use when needed)

// Week 2: Unlearn OOP habits
// ☐ Stop creating interface-per-implementation
// ☐ Stop creating /models, /services, /repositories folders
// ☐ Define interfaces where they are consumed

// Week 3: Embrace Go idioms
// ☐ Use table-driven tests
// ☐ Use context.Context as first parameter
// ☐ Return (result, error) from every fallible function

// Week 4: Think in Go
// ☐ Use channels for goroutine communication
// ☐ Use composition instead of inheritance
// ☐ Keep packages flat and focused
// ☐ Let main() be the composition root
```

65.4 — Complete Series: Phase Map

Phase	Topics Covered
-------	----------------

Phase 1: Syntax & Fundamentals	15 lessons – packages, types, functions, control flow, slices, maps, structs, interfaces, errors, defer
Phase 2: Type System & Patterns	9 lessons – interface composition, generics, struct tags, reflection, functional options, DI, repository pattern
Phase 3: Concurrency	9 lessons – goroutines, channels, select, sync, context, patterns, race detector, atomics, errgroup
Phase 4: Standard Library	9 lessons – net/http, encoding/json, io, testing, database/sql, templates, file I/O, slog, time
Phase 5: Toolchain	6 lessons – modules, build, cross-compile, linting, workspaces, Docker
Phase 6: Advanced	11 lessons – memory model, GC, profiling, unsafe, cgo, gRPC, middleware, errors, generics, reflection, observability
Phase 7: Mindset Shifts	6 lessons – SOLID in Go, no ORM, error verbosity, package design, composition, Go proverbs

TIP: You now have 65 lessons covering the complete journey from C#/.NET to Go. The biggest shift is not syntax — it is philosophy. Go rewards simplicity, explicitness, and composability. Trust the process: after 3 months of writing Go, you will wonder why you ever needed so many abstractions.